1. Computational Tool **01A**

a) 在本节中我们重温了欧几里得的尺规计算机及对应的算法，你还能想到哪些计算及对应的工具？

b) 其中对应的算法都是以什么形式描述的？与 C++语言有何相似与不同之处？

2. Computational Tool **01A**

同样是在船沿刻线做记号，为什么曹冲横着刻就是算法，而竖着刻（刻舟求剑）却不是？

试用算法的概念做出解释。

3. Finiteness **01A**

本节告诉我们程序未必就是算法，比如，我们时常遇到的“死循环”就不满足有穷性。

a) 回顾你的编程经历，还记得自己写过的这类程序吗？

b) 为尽可能避免这类问题，编程及调试过程中你有什么经验？

c) 能否设计一个算法，来自动检测任何程序中是否存在死循环，从而彻底根除？

4. Failstone **01A**

本节指出，至今仍未证明，Hailstone 序列的长度对任何自然数n都有限。现考查如下函数：

$$failstone(p,q)=\begin{cases}1 & (p=q)\\ 1+failstone(p-q,p) & (p>q)\\ 1+failstone(2p,q) & (p<q\ and\ q\ is\ odd)\\ 1+failstone(p,q/2) & (p<q\ and\ q\ is\ even)\end{cases}$$

a) 试编写一个程序，对任何自然数p和q，计算出$failstone(p,q)$（假定字宽足够，不致溢出）；

b) 是否对任何p和q，都有$failstone(p,q)<\infty$？换言之，你的程序能否称作算法？

5. Geometric Distribution **01A**

在本节中我们介绍了“几何分布”，你还见过哪些计算的过程也符合这种分布？

6. Algorithm Performance **01B1**

本节我们以“最小三角形面积”问题为例说明了，即便对于规模接近甚至完全相同的问题实例，有些算法的性能也会差异极大。现在再考查“整数素因子分解”问题的蛮力算法：

- 对任何待分解的整数n，依次用2、3、5、7、11、…做整除测试；
- 每当发现一个素因子d，便输出d，同时将n替换为n/d；
- 直到n等于1

a) 当$n\approx10^{100}$时，该算法最坏情况下需要做多少次整除测试？

b) 最好情况呢？

7. Turing Machine **01B1**

a）对照本节讲义及对应的演示，了解图灵机的结构组成及工作过程；

b) 阅读 increase()算法，掌握其原理；

c) 尝试编写一个 decrease()算法，实现二进制数字串的递减。

8. Yet Another Turing Machine **01B1**

讲义中介绍的图灵机，转换函数要求读写头每次都必须移动，这也是图灵最初设计的版本。

a) 不然理解，如果计算终止时读写头必须复位，那么读写头的累计移动次数必然是偶数。

后来有人做了调整：转换函数中的移动项除了 L 和 R，还允许是 N——读写头可以停在原位不做移动。

b) 试验证：若有某个转换函数的移动项是 N，那么必然需要修改当前单元的内容，或改变读写头的状态。

c) 经此调整之后，图灵机的时间成本应当如何度量？有无本质变化？

9. RAM **01B2**

a ）对照01B3节讲义及对应的演示，了解 RAM 模型的结构组成及工作过程；

b ）阅读 ceiling()算法，掌握其原理；

c) 实现 multiply()算法，实现整数乘法，并估计其时间复杂度；

d ）尝试编写一个 floor()算法，实现向下取整的除法，并估计其时间复杂度；

e ）图灵机与 RAM 之间有哪些区别？

f) 从编程语言的角度看，RAM 与 C++有哪些区别？

10. Small-o **01C1**

01C1节介绍过“大$\mathcal{O}$”和“大Ω”记号，请查阅资料自学了解“小$\mathcal{O}$”和“小ω”记号的含义及作用。

11. Fundamental Calculations **01C2**

讲义上指出，我们将 RAM 中所有基本操作的时间成本都视作$\mathcal{O}(1)$，比如加法、减法。那么，乘法、除法呢？

12. Polynomial Logarithm **01C2**

复杂度函数如果满足$T(n) = \mathcal{O}(f(n))$，是否就必然有$\log(T(n)) = \mathcal{O}(\log(f(n)))$？反过来呢？

13. Exponential Complexity **01C3**

本节通过2-Subset 这个问题来说明，指数复杂度的问题及算法普遍存在。你还能再举出哪些这类例子？

14. Back-Of-Envelope **01D1**

32位、64位最大的无符号整数，表示为10进制是多少位？

15. Back-Of-Envelope **01D1**

a) 阅读 OJ-book 和网络学堂上的《编程作业指引》，熟悉实验平台的硬件配置、编译选项以及作业规范；

b) 在你开始着手做每一道 PA 题之前，参照所设定的时限估计出可行算法的时间复杂度上界。

16. Iteration **01D2**

《习题解析》1-32列举了多种典型的迭代、调用模式，试独立地估计它们的渐近复杂度，并与答案对照。

17. Fractional Series **01D3**

级数之于算法分析的作用，在于估计出计算的成本，比如运行的时间。具体来说，在 RAM 等计算模型中，这类成本都统一度量为基本操作的次数。你应该注意到了，我们温习的级数中不少都是分数形式（比如调和级数），它们对于度量本身为整数的操作次数有什么作用呢？

18. Harmonic Series **01D3**

温习此前在微积分课堂学习的调和级数，确认$\sum_{k=1}^{n} 1/k = \mathcal{O}(\log n)$。

19. Stirling Approximation **01D3**

温习此前在微积分课堂学习的 **Stirling** 近似，确认$\log(n!) = \mathcal{O}(n \log n)$。

20. Storage Cost Of Recursive Algorithms **01E1 + 01E2**

01E1、01E2节分别基于减治、分治的策略，给出了两种递归的 sum()算法，并分析了时间复杂度。

所谓空间复杂度，是指除去输入数据本身后，计算过程所需的空间量。

a) 上述算法的空间复杂度分别是多少？

b) 原本朴素的迭代算法呢？

c) 一般地，递归算法的空间复杂度与哪些因素有关？其中最主要的有哪些？

d) 同一算法的时间、空间复杂度之间，有什么联系？

21. reverse() **01E1**

我们看到，本节的 reverse()算法无非是先做一次 swap，再递归调用一次 reverse。

a) 试确认：即便这两行代码（这两步操作）颠倒次序，算法的功能依然正确；

b) 如此颠倒之后，算法的时间复杂度会否有所差别？如果有，更好了还是更差了？

c) 空间复杂度呢？

22. Master Theorem **01E2**

试证明 **Master** 定理。

23. Regularity Condition **01E2**

Master 定理的第一种情况，附加了一个正则条件：

存在常数$c < 1$，使得对于足够大的n总有 $a \cdot g(n/b) \leq c \cdot g(n)$

a) 试证明，这个条件其实已经蕴含了第一种情况原本的条件：$g(n) = \Omega(n^{\log_b a + \epsilon})$

（因此，原本的条件依然列出，只是为了便于与前两种情况相互对照）；

b) 试举实例说明，两个条件不能反向蕴含。

24. Master Theorem **01E2**

若函数$g()$的形式可以简化为$g(n) = \Theta(n^k \cdot \log^p n)$，则 Master 定理的分类也可以简化。

a) 试证明，此时的三种情况，对应于a与b^k的大小关系，具体地：

- 若$a < b^k$，则 $T(n) = \mathcal{O}(n^k \cdot \log^p n)$
- 若$a > b^k$，则 $T(n) = \mathcal{O}(n^{\log_b a})$
- 若$a = b^k$，则 $T(n) = \mathcal{O}(n^k \cdot \log^{p+1} n)$

b) 第一种情况，是否仍然需要满足正则条件？

25. Akra-Bazzi Theorem **01E2**

试证明 **Akra-Bazzi** 定理。

26. Greatest Slice **01E3**

在本节中，我们介绍了效率由低到高的四种总和最大区间算法。

a) 如果不仅是算出最大总和，还需要具体地给出对应区间的（起始）位置，则应对算法作何调整？

b) 在最大总和同时出现于多个区间的情况下，应如何约定以消除这类歧义？

c) 针对你所做的约定，算法还需要作何调整？

d) 调整后算法的时间、空间复杂度，相对于与原算法是否有所增加？能否保持不变？

27. GS by Divide-And-Conquer **01E3**

基于分治策略设计的总和最大区间算法，选用中点来划分子任务，从而由主定理可知其复杂度为$\mathcal{O}(n \log n)$。

a) 如果改为总是按25%：75%来划分子任务呢？

b) 一般地，对于任何常数$0 < \lambda < 1$，如果总是按λ：$1-\lambda$来划分呢？

28. Memoization **01F1**

翻出你以前编写过的一两个递归程序，试着借助记忆化技巧降低其复杂度。

29. Longest Common Subsequence **01F2**

在本节中我们采用不同策略，分别给出了几种最长公共子序列的算法。

a) 如果不仅要算出 LCS 的长度，还要其中各字符在输出序列中的位置，则应对原算法作何调整？

b) 调整后算法的时间、空间复杂度，相对于与原算法是否有所增加？能否保持不变？

30. Top-Down vs. Bottom-Up 01F1 + 01F2

本节所实现的两种最长公共子序列算法，都是基于子问题之间的同一套递推关系，因此二者在本质上是等价的，其差异仅在于各自的求解方向：如果说记忆化版是自顶而下，那么动态规划版就是自底而上。

a) 就最长公共子序列这个问题而言，两种方式相对而言各有什么优点？

b) 对于其他也可以采用这两种方式来求解的问题，你的上述结论是否依然成立？

31. Code Reading

a) 下载本课程提供的示例代码包，在 Visual Studio 中通过 dsacpp.sln 打开整个解决方案；

b) 找出本章所涉及的项目，分别编译、运行，观察运行结果，并细读对应的代码。

32. Demo

a) 下载算法演示包，并参照网络学堂公告中所介绍的方法，学会使用 excel 和 applet 类型的演示；

b) 找到本章所涉及的演示，对照讲义经常把玩，反复推敲。